SC4003 Assignment 1 Report

March 4, 2026

1 Problem setup

The base maze used in Part 1 is shown in Figure 1. The discount factor is fixed at 0.99, exactly as required by the assignment.

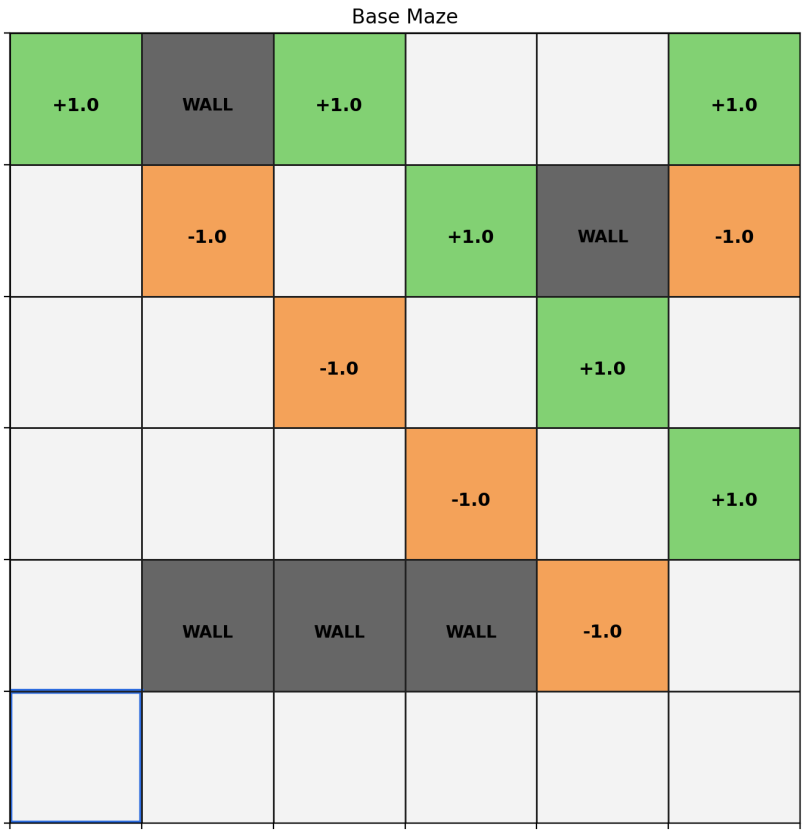

Figure 1: Assignment maze used for Part 1. Green cells have reward +1, brown cells have reward -1, white cells have reward -0.05, and gray cells are walls.

2 Using method of value iteration for Part 1

2.1 Descriptions of implemented solutions

The assignment maze is a 6x6 grid with 5 walls, 6 green reward cells (+1), 5 brown reward cells (-1), and white cells with reward -0.05. The agent has four actions: up, right, down, and left. The intended move happens with probability 0.8, and the two perpendicular moves each happen with probability 0.1. If a move hits a wall or the boundary, the agent stays in the same

state. A crucial detail from the assignment is that there are no terminal states, so green and brown cells remain revisitable. Value Iteration updates every non-wall state with the Bellman optimality equation $U_{k+1}(s) = R(s) + \gamma \max_a \sum_{s'} P(s' \mid s, a)\, U_k(s')$ until the change between two consecutive sweeps is below the stopping threshold.

2.2 Plot of optimal policy

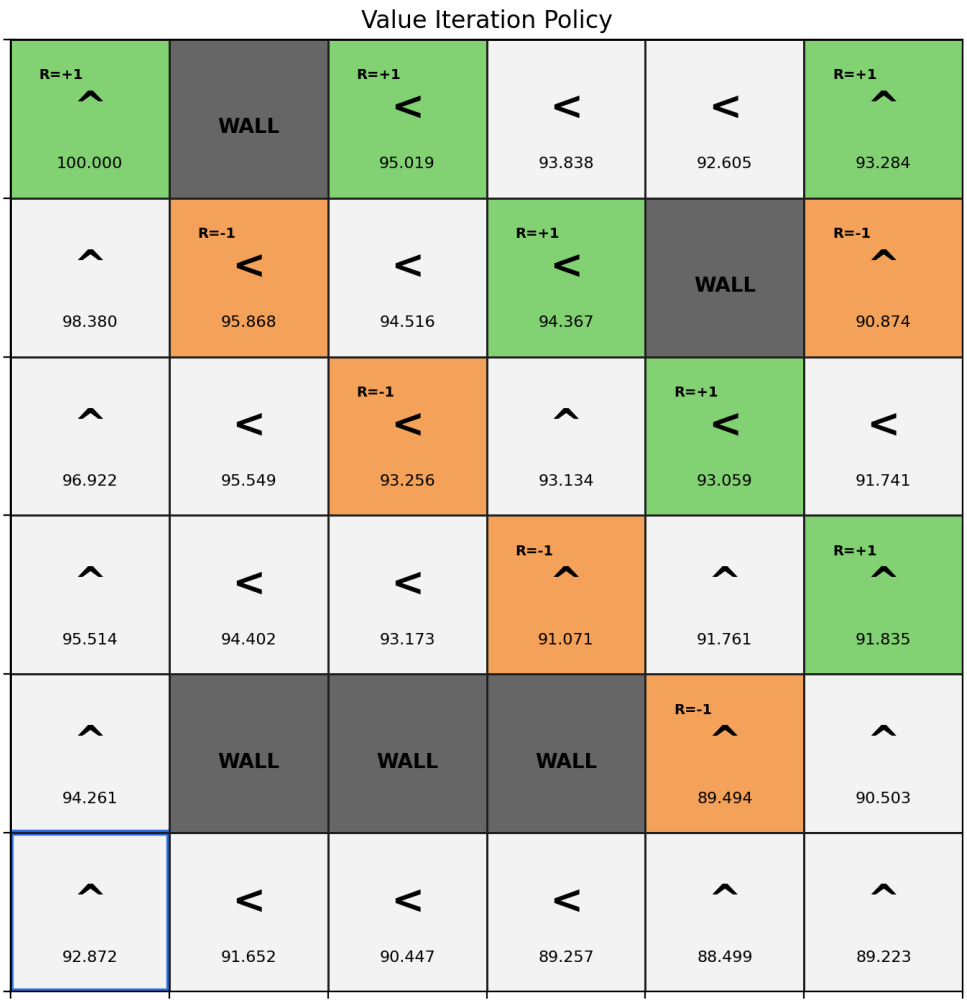

Figure 2: Optimal policy produced by Value Iteration.

The policy points toward repeatedly collecting high reward while avoiding paths that make accidental moves into brown cells more likely. This matches the assignment requirement to display the optimal policy for all non-wall states.

2.3 Utilities of all states

These utilities are much larger than the immediate rewards because the assignment uses an infinite-horizon discounted process with no terminal states. A state is valuable not only for its next reward, but also for how well it can lead the agent back to green cells repeatedly.

2.4 Plot of utility estimates as a function of the number of iterations

The curves flatten out near the end of the plot, which is the practical sign that the utility estimates have converged.

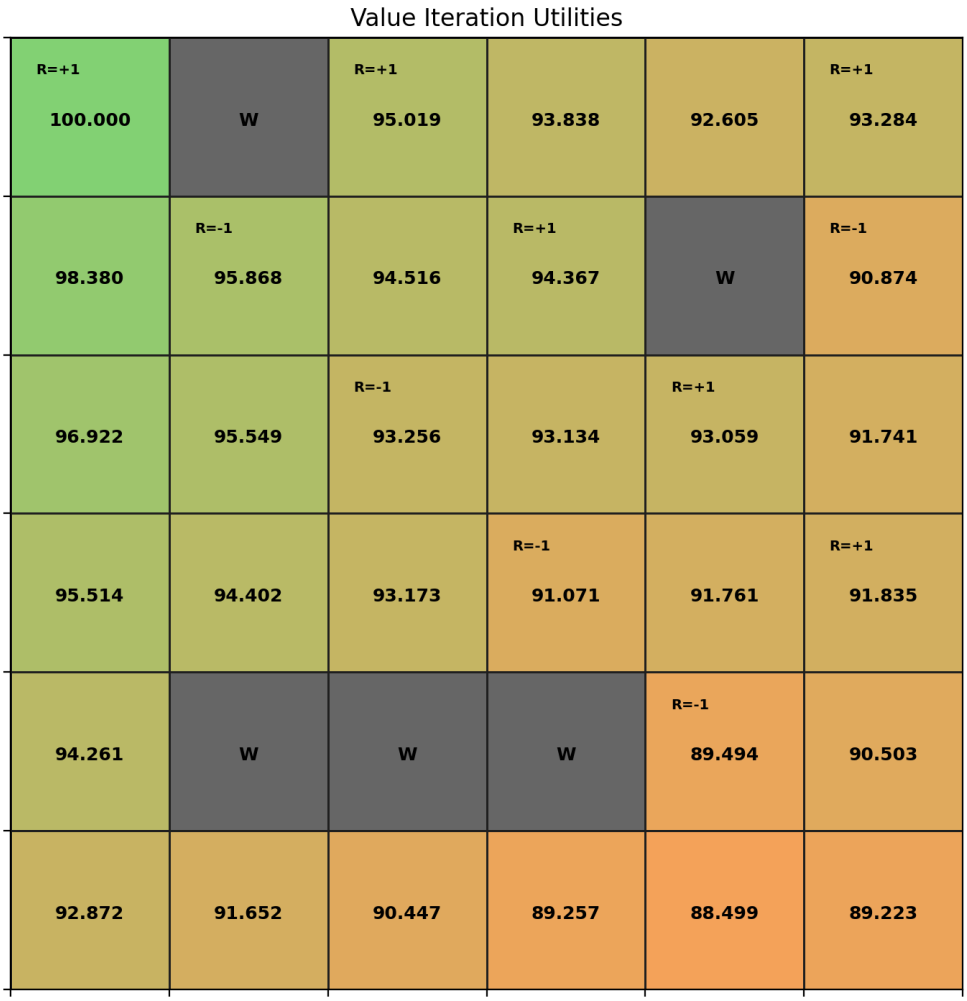

Figure 3: Utility heatmap for Value Iteration. Each number is the utility of a non-wall state. Green and brown reward cells are revisitable states, not terminal states.

3 Using method of policy iteration for Part 1

3.1 Descriptions of implemented solutions

Policy Iteration uses the same MDP model as Value Iteration. It starts from an initial policy, repeatedly evaluates the current policy, and then improves it greedily using the updated utilities. Policy evaluation is implemented by iterative sweeps using $U^{\pi}(s) = R(s) + \gamma \sum_{s'} P(s' \mid s, \pi(s))\, U^{\pi}(s')$, and policy improvement replaces the current action with the action that gives the highest expected return. Because there are no terminal states, the policy is defined for every non-wall cell, including green and brown reward cells.

3.2 Plot of optimal policy

The final policy produced by Policy Iteration matches the one from Value Iteration. This agreement is expected because both algorithms solve the same discounted MDP.

3.3 Utilities of all states

The utility magnitudes are almost identical to those from Value Iteration, which confirms that the two methods converged to the same solution for Part 1.

99.999900	WALL	95.019451	93.837684	92.605293	93.283697
98.380476	95.867564	94.516375	94.367165	WALL	90.873675
96.921948	95.548882	93.255956	93.134004	93.059302	91.741117
95.513866	94.402131	93.172948	91.070553	91.760608	91.834921
94.260574	WALL	WALL	WALL	89.494111	90.502967
92.872268	91.651914	90.446777	89.256666	88.498801	89.222786

Table 1: Utility values for Value Iteration.

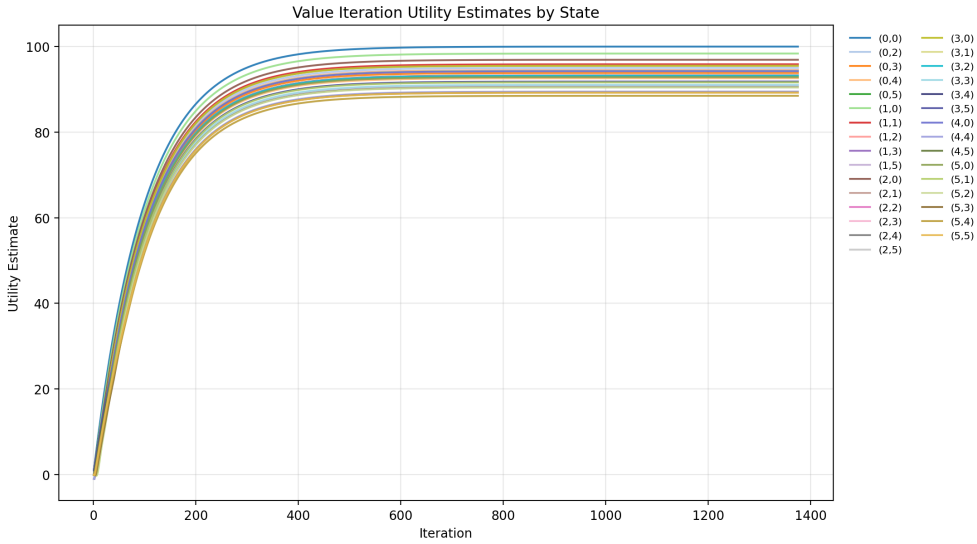

Figure 4: Utility estimate of every non-wall state versus iteration for Value Iteration. Total iterations: 1375. Runtime: 317.235 ms.

3.4 Plot of utility estimates as a function of the number of iterations

The curves flatten out near the end of the plot, which is the practical sign that the utility estimates have converged.

4 Source code for Part 1

- Environment model: `src/gridworld.py`
- Value Iteration: `src/value_iteration.py`
- Policy Iteration: `src/policy_iteration.py`
- Visualization: `src/visualization.py`
- CLI entrypoint: `main.py`

The code follows the report structure directly: the environment model defines the MDP, the two algorithm files implement Value Iteration and Policy Iteration, and the plotting/report code exports the policy plots, utility tables, and convergence figures required in the assignment.

5 Part 2 questions

5.1 Complicated maze design

For Part 2, I designed a family of larger mazes instead of changing only one map manually. Starting from size 7 and increasing to size 17, each generated maze contains more white states,

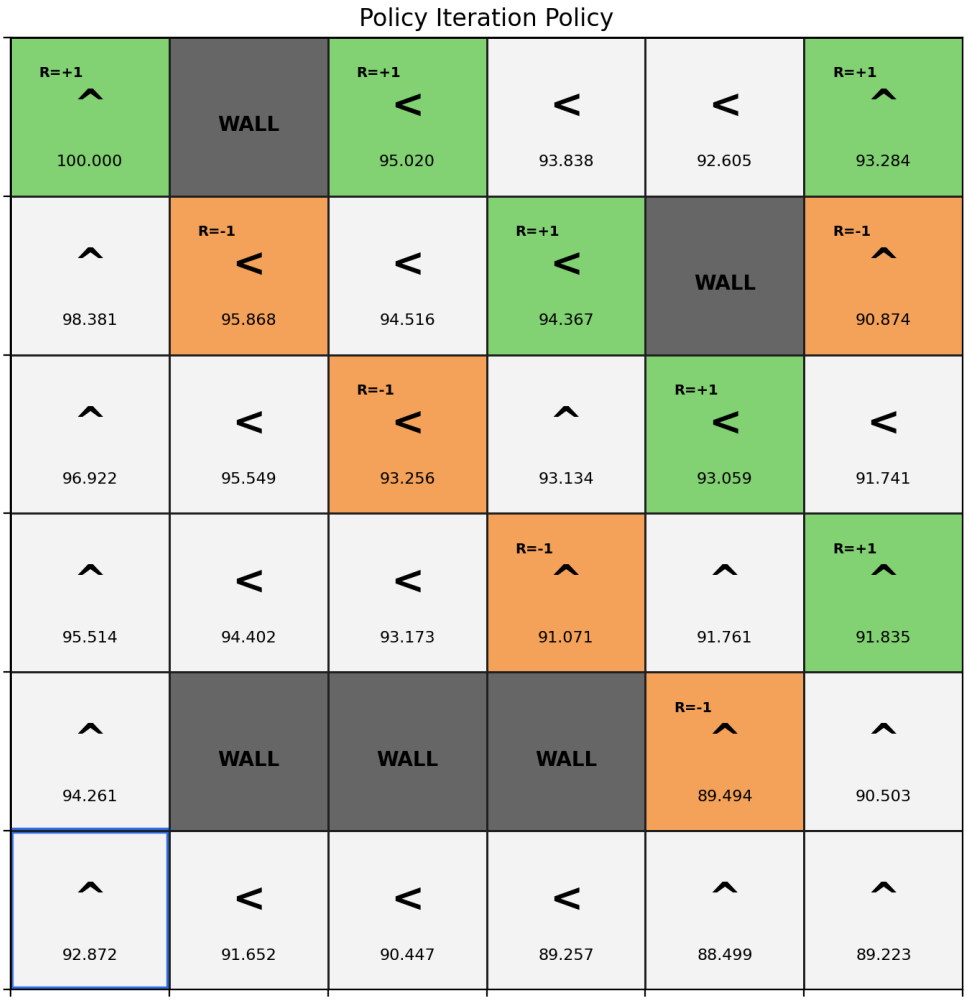

Figure 5: Optimal policy produced by Policy Iteration.

more walls, and more positive/negative reward cells. The same transition model as Part 1 is kept, and the random seed is fixed at 42 so that the experiments are reproducible.

5.2 How do the number of states and the complexity of the environment affect convergence?

Table 3 and the plots below show a clear trend: when the maze gets larger, both algorithms need more computation before the utilities stabilize. Value Iteration usually needs more repeated Bellman sweeps over the whole state space. Policy Iteration often reaches a stable policy with a different update pattern, but its evaluation stage is more expensive per round, so its runtime is not automatically lower.

5.3 How complex can the environment be while still learning the right policy?

In this implementation, both algorithms still converged and produced the same final policy for every tested maze size from 7 to 17. Therefore, within the tested range, the largest environment that still learned a matching optimal policy was the 17x17 maze. The policy mismatch column in Table 3 is zero for every run, which is the main evidence used here.

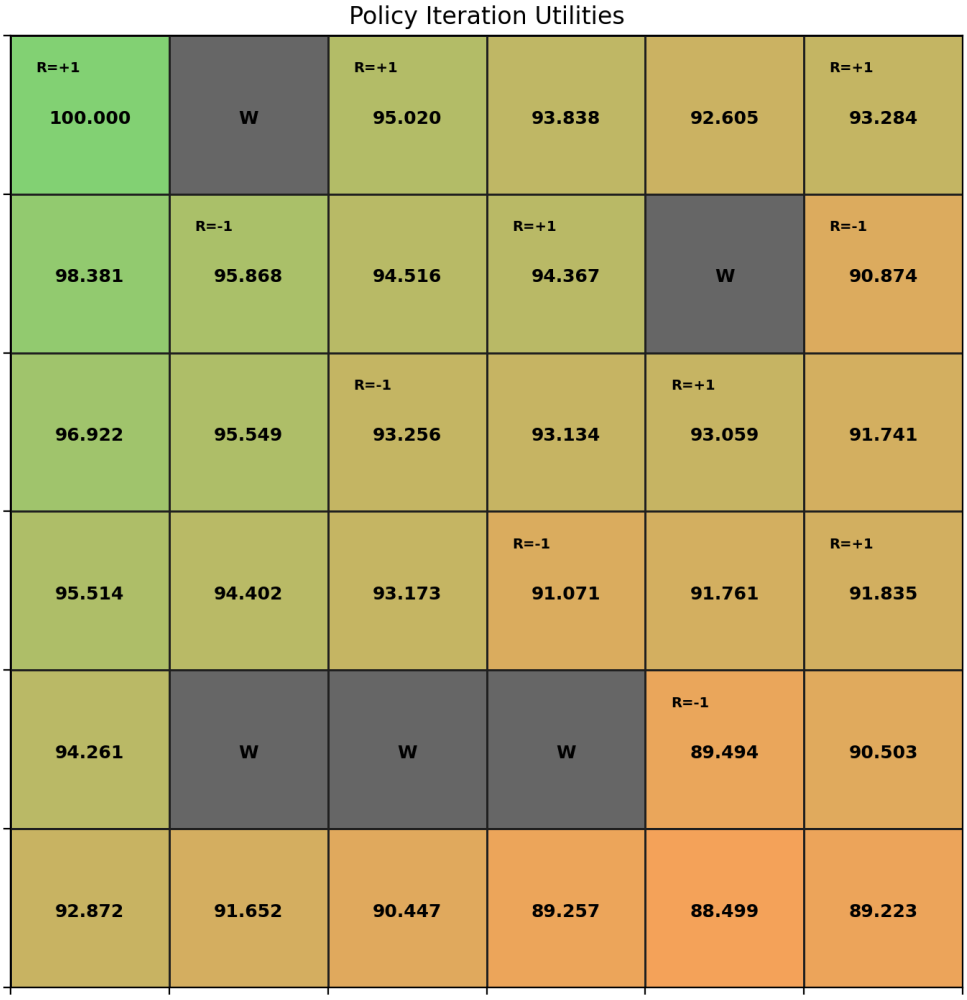

Figure 6: Utility heatmap for Policy Iteration. Each number is the utility of a non-wall state. Green and brown reward cells are revisitable states, not terminal states.

100.000000	WALL	95.019551	93.837784	92.605393	93.283795
98.380576	95.867664	94.516475	94.367265	WALL	90.873773
96.922048	95.548981	93.256056	93.134104	93.059401	91.741216
95.513965	94.402230	93.173048	91.070652	91.760707	91.835020
94.260674	WALL	WALL	WALL	89.494210	90.503066
92.872368	91.652014	90.446876	89.256766	88.498900	89.222885

Table 2: Utility values for Policy Iteration.

Size	VI Iter	PI Iter	VI Runtime	PI Runtime	VI Mean Utility	Mismatch
7	1375	2504	355.357	172.005	91.438447	0
9	1361	3399	627.222	405.463	78.059848	0
11	1361	2801	967.846	555.94	75.173058	0
13	1375	2935	1425.138	824.995	84.911037	0
15	1361	5098	2032.416	2074.456	73.839851	0
17	1361	5526	2831.517	2765.528	73.081162	0

Table 3: Part 2 comparison across generated mazes.

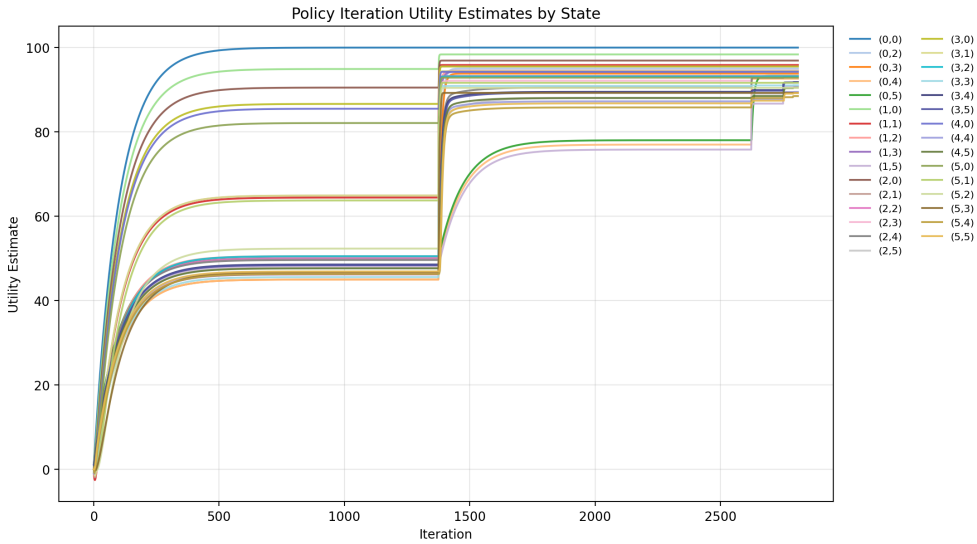

Figure 7: Utility estimate of every non-wall state versus iteration for Policy Iteration. Total iterations: 2809. Runtime: 170.041 ms.

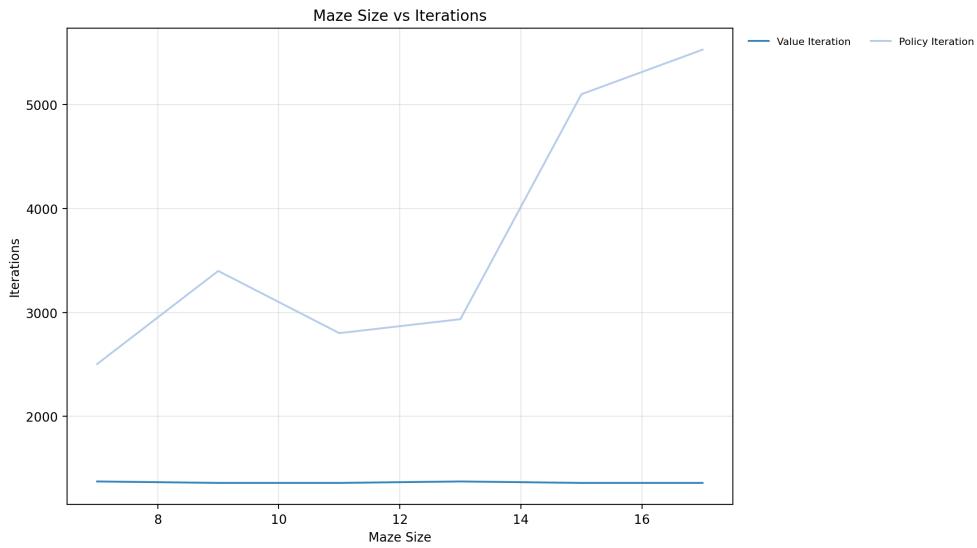

Figure 8: Iterations required by each method as maze size increases.

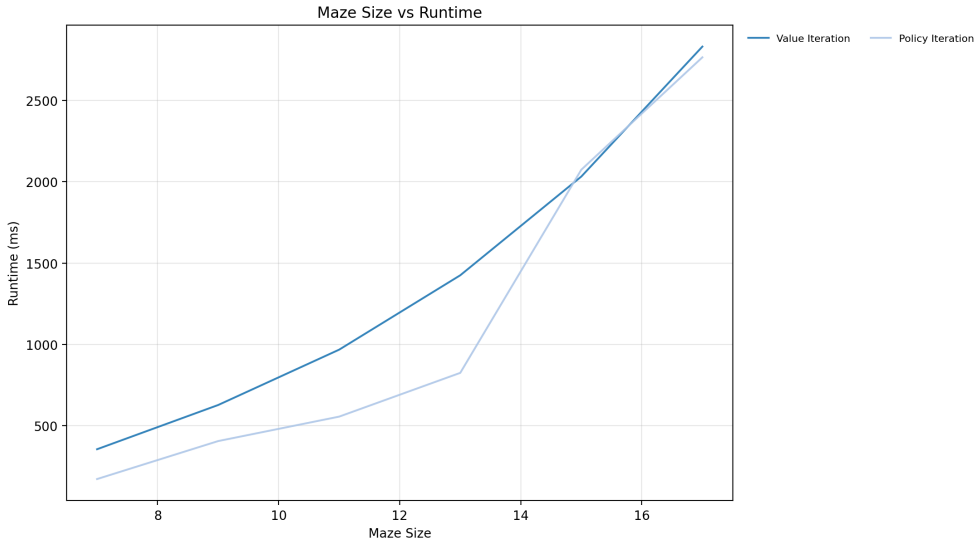

Figure 9: Runtime comparison as maze size increases.

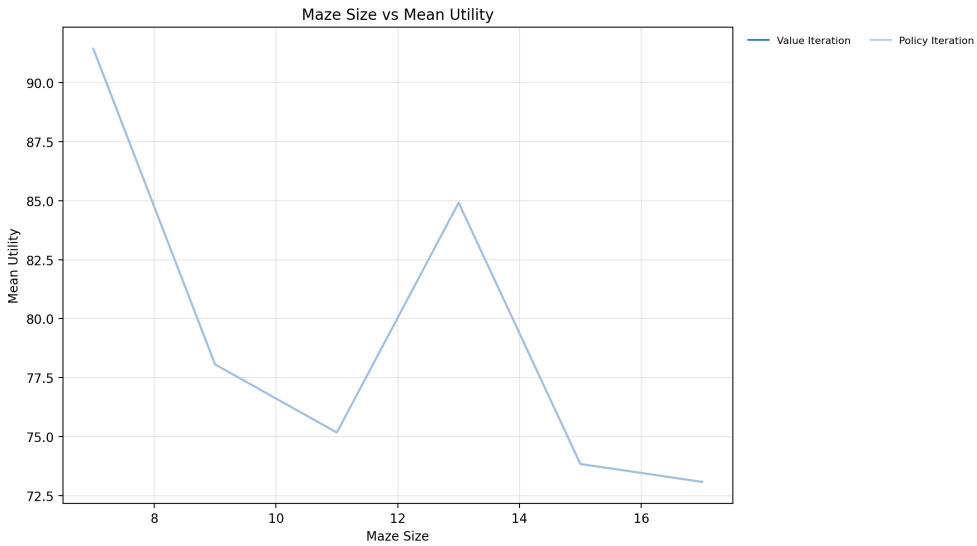

Figure 10: Mean utility comparison as maze size increases.